



OKX Smart Wallet (EVM)

Security Audit Report

Prepared by

OKX Web3 Audit Team

15 March 2026

Contents

1	Overview	3
1.1	Project Introduction	3
1.2	Audit Summary	3
1.3	Audit Scope	3
2	Audit Summary	4
2.1	Audit Methodology	4
2.2	Audit Process	4
2.3	Risk Classification and Description	5
2.4	Results	5
3	Vulnerabilities	7
3.1	Low - Chainless UserOp Does Not Validate Admin Permission, Wasting Bundler Gas	7
3.2	Low - isValidSignature() Implementation Does Not Comply with EIP-1271	9
4	Disclaimer	11
5	About OKX Web3 Audit Team	11

1. Overview

1.1 Project Introduction

OKX Smart Wallet is an EVM-based smart contract wallet implementing ERC-4337 account abstraction. It supports multiple owner types (ECDSA and Passkey), chainless UserOps, and EIP-7702 authorization for delegated execution.

1.2 Audit Summary

Ecosystem	EVM
Language	Solidity, Rust
Repository	https://github.com/okx/okx-smart-wallet/
Base Commit	c3211b52566f998b221ac4dad915ecf54c3d4da3 (origin/dev)
Final Commit	c24fe65e1d813ca98dfece66fab660f2b683e4f9

1.3 Audit Scope

```
okx-smart-wallet/src/  
└─ *.sol
```

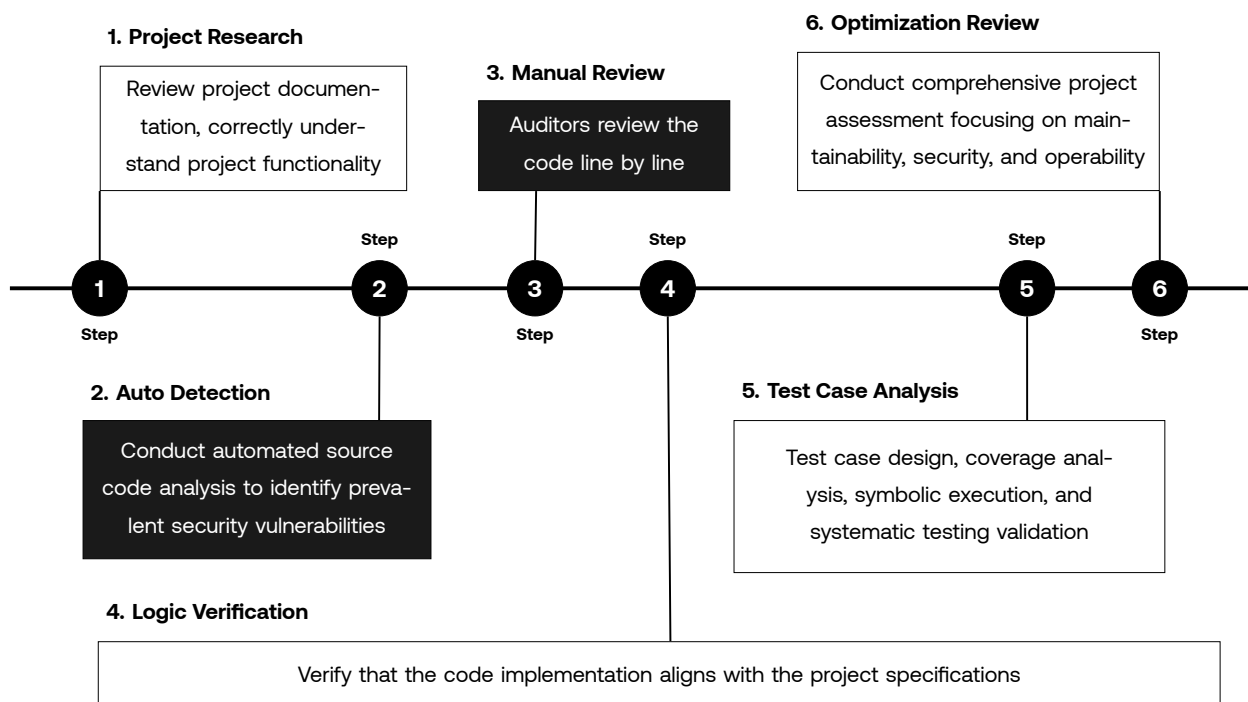
2. Audit Summary

2.1 Audit Methodology

The audit team conducted comprehensive analysis of the contract code through deep understanding of the project's design purpose, operating principles, and implementation methods. By mapping function call relationships, potential security vulnerabilities were systematically identified, with detailed problem descriptions and corresponding remediation recommendations provided.

2.2 Audit Process

The smart contract security audit follows a 6-phase process: Project Research, Automated Detection, Manual Review, Logic Verification, Test Case Analysis, and Optimization Review. During manual auditing, auditors perform comprehensive code review to identify vulnerabilities and provide detailed solutions. After completing all phases, the lead auditor communicates findings with the project team. Following the team's responses, we deliver final audit reports to the project team.



2.3 Risk Classification and Description

Risk items are classified into 5 levels: Critical, High, Medium, Low, and Informational. Critical risks require immediate resolution and re-audit before final report delivery; unresolved critical risks result in audit failure.

Risk Level	Risk Description
Critical	Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
High	High risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.
Medium	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
Low	Low risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
Informational	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

2.4 Results

The audit results are divided into two parts: one part is the vulnerability summary of the project audit, and the other part is the detailed vulnerability list.

Vulnerability Summary

Critical	High	Medium	Low	Informational	Total
0	0	0	2	0	2

Vulnerability list

No.	Severity	Vulnerability	Category	Status
1	Low	Chainless UserOp Does Not Validate Admin Permission, Wasting Bundler Gas	Logic Error	Confirmed
2	Low	isValidSignature() Implementation Does Not Comply with EIP-1271	EIP Non-compliance	Confirmed

Status Definitions

- **Open:** The audit team has notified the project team of the vulnerability, but no reasonable remediation has been implemented.
- **Fixed:** The project team has addressed the vulnerability and the fix has been verified by the audit team.
- **Confirmed:** The project team has confirmed awareness of the vulnerability risk but considers it controllable.

3. Vulnerabilities

This section outlines the risk items identified through manual review and auditing tools. Each item includes the specific file path and code location, along with the assigned risk level.

3.1 Low - Chainless UserOp Does Not Validate Admin Permission, Wasting Bundler Gas

Location	File	Category	Status	Severity
Line 274	SmartWallet.sol	Logic Error	Confirmed	Low

Description

The chainless path in `validateUserOp()` only validates whether the called selector is in the allowlist (`addOwner`, `upgradeToAndCall`), but does not check whether the signer holds admin permission. The functions in the chainless allowlist can only be called by an admin through `_batchCall`. Because `validateUserOp` omits this check, the following scenario arises:

1. A non-admin owner signs a chainless UserOp.
2. `validateUserOp` returns success (valid signature, selector in allowlist).
3. `EntryPoint` executes `executeUserOp`.
4. `_batchCall` reverts with `NonAdminSelfCall`.
5. The bundler loses gas (validation passed, execution failed).

Related Code

```
270 function validateUserOp(  
271     PackedUserOperation calldata userOp,  
272     bytes32 userOpHash,  
273     uint256 missingAccountFunds  
274 ) external onlyEntryPoint returns (uint256 validationData) {  
275     // ...  
276  
277     // Step 5: Handle chainless execution if applicable  
278     uint256 nonceKey = userOp.nonce >> 64;  
279     if (nonceKey == Static.CHAINLESS_NONCE_KEY) {  
280         // Decode calls from userOp.callData  
281         Call[] calldata calls = DecodeLib.decodeCalls(userOp.callData[4:]);  
282     }
```

```
283      // @audit: No admin permission check here. A non-admin chainless UserOp
284      // passes validateUserOp but reverts in executeUserOp -> _batchCall
285      // with NonAdminSelfCall, wasting bundler gas.
286      if (
287          !ChainlessLib.validateChainlessNonceCallData(
288              calls,
289              address(this)
290          )
291      ) {
292          return Static.SIG_VALIDATION_FAILED;
293      }
294
295      userOpHash = getUserOpHashWithoutChainId(userOp);
296  }
297  // ...
298 }
```

Recommendation

Add an admin permission check in the chainless path of `validateUserOp()` to reject UserOps signed by non-admin owners before they reach the execution phase. This ensures that a chainless UserOp targeting admin-only functions fails at validation rather than at execution, preventing bundler gas loss.

Project Team Feedback

Team Response	No modification. The <code>isAdmin</code> permission check is already centralized in the <code>_batchCall</code> method. Adding a duplicate check in <code>validateUserOp</code> would scatter the permission logic and reduce code clarity.
Re-audit Result	N/A

3.2 Low - isValidSignature() Implementation Does Not Comply with EIP-1271

Location	File	Category	Status	Severity
Line 32, Line 48	ECDSAValidatorLib.sol, PasskeyValidatorLib.sol	EIP Non-compliance	Confirmed	Low

Description

The `validateSignature()` functions in `ECDSAValidatorLib` and `PasskeyValidatorLib` may revert when `abi.decode` fails due to malformed `validatorData`. This revert propagates to the `SmartWallet` contract's `isValidSignature()` function, causing it to revert as well.

This violates EIP-1271, which requires `isValidSignature()` to always return a value and never revert. External contracts relying on EIP-1271 behavior (e.g., calling `isValidSignature()` and expecting a `bytes4` return value) may fail or behave unexpectedly when the function reverts instead.

Related Code

```
32 function validateSignature(  
33     bytes32 keyHash,  
34     bytes32 messageHash,  
35     bytes calldata validatorData  
36 ) internal pure returns (bool) {  
37     if (validatorData.length < ECDSA_SIGNATURE_LENGTH) {  
38         return false;  
39     }  
40  
41     bytes memory signature = validatorData[:ECDSA_SIGNATURE_LENGTH];  
42     if (validatorData.length > ECDSA_SIGNATURE_LENGTH) {  
43         bytes32[] memory proofs = abi.decode(  
44             validatorData[ECDSA_SIGNATURE_LENGTH:],  
45             (bytes32[])  
46         ); // @audit: may revert on malformed input, violating EIP-1271  
47         messageHash = MerkleProofProcessor.processWithMerkleProof(  
48             proofs,  
49             messageHash  
50         );  
51     }  
52  
53     (address recoveredSigner, , ) = messageHash.tryRecover(signature);  
54     return keccak256(abi.encodePacked(recoveredSigner)) == keyHash;  
55 }
```

Recommendation

Wrap the `abi.decode` call in a try/catch block (or use a low-level approach) in `ECDSAValidatorLib` and `PasskeyValidatorLib` so that malformed input returns false instead of reverting. This ensures `isValidSignature()` always returns a value, conforming to EIP-1271.

Project Team Feedback

Team Response	No modification. Since the decode process is complex, reverting on unexpected input format is considered normal and acceptable behavior.
Re-audit Result	N/A

4. Disclaimer

This audit report only covers the specific audit types stated herein. We assume no responsibility for unknown security vulnerabilities outside this scope.

We rely on audit reports issued before existing attacks or vulnerabilities are published. For future or new vulnerabilities, we cannot guarantee project security impact and assume no responsibility.

Our security audit analysis should be based on documents provided by the project before report release (including contract code). These materials should not contain false information, tampering, deletion, or concealment. If provided materials are false, inaccurate, missing, tampered, deleted, concealed, or modified after report release, we assume no responsibility for resulting losses and adverse effects.

The project team should understand our audit report is based on provided materials and current technical capabilities. Due to institutional technical limitations, our report may not detect all risks. We encourage continued testing and auditing by the development team and stakeholders.

The project team must ensure compliance with applicable laws and regulations.

The audit report is for reference only. Its content, acquisition methods, usage, and related services cannot serve as basis for investment, taxation, legal, regulatory, or construction decisions. Without our prior written consent, the project team may not reference, cite, display, or distribute report content to third parties. Any resulting losses shall be borne by the project team.

This report does not cover contract compiler bugs or scope beyond programming languages. Smart contract risks from underlying vulnerabilities should be borne by the project team.

Force majeure includes unforeseeable, unavoidable events like wars, natural disasters, strikes, epidemics, and legal/regulatory changes preventing contract performance. When occurring, neither party breaches contract obligations. For unaffected economic responsibilities, the project team should pay for completed work.

5. About OKX Web3 Audit Team

OKX Web3 Audit Team specializes in blockchain security with expertise in smart contract auditing, token security assessment, and Web3 security tool development. We provide comprehensive security solutions for OKX's internal Web3 projects, conduct pre-listing token audits, and develop security tools to protect OKX Web3 wallet users. Our team combines automated analysis with manual review to deliver thorough security assessments and maintain the highest security standards in the Web3 ecosystem.